

(a) Python Implementation

Note on Terminology:

"Square-block distance" is another term for **Manhattan distance** or **City Block distance**, calculated as the sum of the absolute differences of their coordinates.

Python

```
import math
```

```
from collections import Counter
```

```
# 1. Setup the Dataset
```

```
dataset = [  
    {'coords': (30, 30, 10), 'class': 'A', 'label': 'P1'},  
    {'coords': (20, 10, 20), 'class': 'A', 'label': 'P2'},  
    {'coords': (10, 20, 20), 'class': 'B', 'label': 'P3'},  
    {'coords': (15, 40, 60), 'class': 'A', 'label': 'P4'},  
    {'coords': (6, 5, 10), 'class': 'B', 'label': 'P5'},  
    {'coords': (20, 10, 15), 'class': 'B', 'label': 'P6'},  
    {'coords': (13, 15, 7), 'class': 'B', 'label': 'P7'},  
    {'coords': (40, 70, 70), 'class': 'A', 'label': 'P8'}  
]
```

```
# The new object to classify
```

```
Z_obj = (10, 30, 5)
```

```
# 2. Distance Functions
```

```
def euclidean_distance(p1, p2):  
    """Calculates the straight-line distance between two points."""  
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(p1, p2)))
```

```
def square_block_distance(p1, p2):  
    """Calculates the Manhattan/City-Block distance."""  
    return sum(abs(a - b) for a, b in zip(p1, p2))
```

```
# 3. The k-NN Algorithm
```

```
def knn_classify(data, target, k=3, metric='euclidean', weighted=False):  
    distances = []
```

```
    # Calculate distance from target to all points in the dataset
```

```
    for item in data:
```

```
        if metric == 'euclidean':
```

```
            dist = euclidean_distance(item['coords'], target)
```

```
        elif metric == 'square_block':
```

```
            dist = square_block_distance(item['coords'], target)
```

```
        distances.append({  
            'label': item['label'],  
            'dist': dist,  
            'class': item['class']  
        })
```

```
    # Sort the list by distance (closest first)
```

```
    distances.sort(key=lambda x: x['dist'])
```

```

# Isolate the k-nearest neighbors
neighbors = distances[:k]

if not weighted:
    # Standard Voting: Count the classes of the neighbors
    votes = [neighbor['class'] for neighbor in neighbors]
    vote_counts = Counter(votes)
    return vote_counts.most_common(1)[0][0] # Return the majority class

else:
    # Distance-Weighted Voting: Closer neighbors get stronger votes
    weights = {'A': 0, 'B': 0}
    for neighbor in neighbors:
        # Weight = 1 / distance. (Added 1e-5 to prevent division by zero if points overlap)
        weight = 1 / (neighbor['dist'] + 1e-5)
        weights[neighbor['class']] += weight

    # Return the class with the highest total weight
    return max(weights, key=weights.get)

```

4. Execution

```

print("Assuming k = 3 for the classification:\n")
print(f"(i) Euclidean Distance Class: {knn_classify(dataset, Z_obj, k=3, metric='euclidean', weighted=False)}")
print(f"(ii) Weighted Euclidean Class: {knn_classify(dataset, Z_obj, k=3, metric='euclidean', weighted=True)}")
print(f"(iii) Square-Block Distance Class: {knn_classify(dataset, Z_obj, k=3, metric='square_block', weighted=False)}")

```
